



C²²MP: the marriage of catch22 and the matrix profile creates a fast, efficient and interpretable anomaly detector

Sadaf Tafazoli¹ · Yue Lu¹ · Renjie Wu¹ · Thirumalai Vinjamoor Akhil Srinivas¹ · Hannah Dela Cruz¹ · Ryan Mercer¹ · Eamonn Keogh¹

Received: 25 December 2023 / Revised: 28 February 2024 / Accepted: 20 March 2024 /

Published online: 11 May 2024

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2024

Abstract

Many time series data mining algorithms work by reasoning about the relationships the conserved *shapes* of subsequences. To facilitate this, the Matrix Profile is a data structure that annotates a time series by recording each subsequence's Euclidean distance to its nearest neighbor. In recent years, the community has shown that using the Matrix Profile it is possible to discover many useful properties of a time series, including repeated behaviors (motifs), anomalies, evolving patterns, regimes, etc. However, the Matrix Profile is limited to representing the relationship between the subsequence's *shapes*. It is understood that, for some domains, useful information is conserved not in the subsequence's *shapes*, but in the subsequence's *features*. In recent years, a new set of features for time series called catch22 has revolutionized feature-based mining of time series. Combining these two ideas seems to offer many possibilities for novel data mining applications; however, there are two difficulties in attempting this. A direct application of the Matrix Profile with the catch22 features would be prohibitively slow. Less obviously, as we will demonstrate, in almost all domains, using all twenty-two of the catch22 features produces poor results, and we must somehow select the subset appropriate for the domain. In this work, we introduce novel algorithms to solve both

✉ Sadaf Tafazoli
stafa002@ucr.edu

Yue Lu
ylu175@ucr.edu

Renjie Wu
rwu034@ucr.edu

Thirumalai Vinjamoor Akhil Srinivas
tsrin002@ucr.edu

Hannah Dela Cruz
hdela004@ucr.edu

Ryan Mercer
rmerc002@ucr.edu

Eamonn Keogh
eamonn@cs.ucr.edu

¹ University of California, Riverside, USA

problems and demonstrate that, for most domains, the proposed C^{22} MP is a state-of-the-art anomaly detector.

Keywords Time series · Anomaly detection · Feature extraction · Matrix profile · Discords

1 Introduction

Many time series data mining algorithms, including motif discovery, summarization and anomaly detection, often essentially reduce to reasoning about conserved *shapes* of subsequences. For example, a subsequence that is very far from its nearest neighbor must be relatively unusual by definition and is therefore a candidate anomaly. Likewise, pairs of subsequences that are mutually close to each other (motifs) are suggestive of some mechanism for conserved behavior, perhaps a gesture in a smartwatch telemetry or a recurring fault in industrial time series. In recent years, the Matrix Profile (MP) has emerged as one of the most promising general tools for time series data mining [54]. The MP is a data structure that annotates a time series by recording each subsequence's distance to its nearest neighbor. Typically, the z-normalized Euclidean distance is used, although a handful of other shape-based measures have been proposed, including Dynamic Time Warping. It has been shown that Matrix Profile allows for discovery of many regularities and structures in the original time series, including motifs, anomalies (discords), evolving patterns (chains), regimes, etc. However, the Matrix Profile has an important and underappreciated limitation: it is limited to representing the relationships between *shapes* of subsequences. For some domains, useful information is conserved not in the subsequence's *shapes*, but in the subsequence's *features*. For example, imagine we have a long typical electrocardiogram that is corrupted by two 1-s bursts of random noise in two locations. Discovering this “motif of noise” could be very useful, as we could correlate it with its cause (perhaps interference from a high frequency ultrasound being used in an adjacent room in the hospital). However, under Euclidean distance the two subsequences will *not* be similar (in fact they will be almost as far apart as the diameter of the space). In such domains, the MP is of little use.

In parallel to the explosion of interest in the MP, there has been a significant breakthrough in time series feature extraction. The issue with feature extraction for time series was its ad-hoc nature. There are several thousands of named time series features in the literature, naturally including many subsets that are highly redundant. Thus, any practitioner hoping to use a feature-based approach was required to find a set of non-redundant features that were appropriate for both the domain and the task-at-hand. In [34], the authors provided a limited features set of twenty-two features, the eponymous *catch22* (C22), to make this task easier. These twenty-two features were discovered by an incredibly ambitious brute force search through a massive set of 4791 candidate features, evaluated on an enormous and diverse set of 147,000 time series datasets. It is no exaggeration to claim that *catch22* has revolutionized feature-based mining of time series. There is a real sense in which it has democratized and deskilled time series data analytics. For example, allowing biologists to study the motion of trees in the wind without the need of data mining specialists [24].

Note that the Matrix Profile and *catch22* are complementary ideas. The MP allows us to reason about the relationships between subsequences. For example, “*These two subsequences are alike*” (motifs [54]) or “*this subsequence is different to all others*” (discords) [4–6, 39], whereas C22 simply summarizes each individual subsequence's properties in a useful and intuitive way. As such, combining these two ideas into a single framework seems to offer many

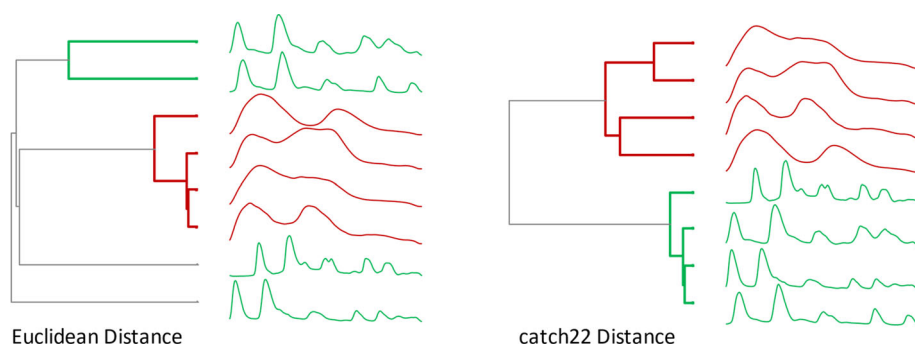


Fig. 1 Eight instances from the 50words dataset clustered using Euclidean distance (*left*) and catch22 (*right*). At least for these two classes, catch22 is better able to represent the conserved class-specific structure

possibilities for novel data mining applications. The utility of doing this is predicated on the assumption that there can be semantic structure in time series that is better conserved in *features* than in *shapes*. In fact, this *is* the case. In Fig. 1, we demonstrate this by clustering exemplars from two classes of the 50words dataset from the UCR archive [11].

However, there are two difficulties in attempting to unify these two ideas. A direct application of the Matrix Profile with the catch22 features would be intractable. The scalability of the SOTA Matrix Profile algorithm, SCRIMP [54], is due to its exploiting several unique properties of the Euclidean distance function. It does not appear that catch22 features are similarly amiable to such optimizations.

Less obviously, in almost all domains, using *all* twenty-two of the catch22 features will produce poor results. While some subset of the features will be sensitive to however an anomaly may reveal itself, it is very likely that there will also be some features that are very sensitive to irrelevant natural variations in the data, and the inclusion of these features will cause false alarms.

In this work, we introduce a novel representation and algorithm, C²²MP and ORR, to solve both problems. We will demonstrate that our algorithm is orders of magnitude faster than a direct application of C22 to long time series, and we will introduce a framework to allow us to discover an appropriate subset of features for the domain. While the C²²MP can be applied to diverse problems, in this work we will confine our attention to just anomaly detection (*discord discovery* in the language of the MP [54]), reserving further generalizations to future work. We will demonstrate that the C²²MP can find subtle anomalies that no other SOTA algorithm can discover.

The rest of the paper is organized as follows: in Sect. 2, we motivate the need for a feature-based anomaly detector before discussing related work in Sect. 3. Section 4 introduces the necessary definitions and notations. In Sect. 5, we first introduce algorithms to compute the C²²MP without regard to the weights; then, we introduce four strategies to set the weights under different assumptions/conditions. Finally, in Sect. 6, we conduct an extensive empirical evaluation before offering conclusions and suggestions for future work in Sect. 7.

2 Motivation

With hundreds of proposed algorithms for Time Series Anomaly Detection (TSAD), it is reasonable to question the need for yet another one. Most of the existing TSAD algorithms found in literature only focus on the *shapes* of subsequences. This is evident in approaches such as Matrix Profile and related techniques [33, 54], and appears to be the case for most deep learning approaches as well [21]. although many deep learning methods are opaque, even to the researchers who introduced them. Reasoning about changes in shape clearly works very well for some data types: gait, heartbeats, gestures, daily traffic patterns, etc. But many datasets may have richer and more complex behaviors, such as the insect behavior shown in Fig. 2.

The insect data is clearly not random, but the conservation of behavior is much more complex than simple repetition of shape, one could almost argue for a complex “grammar” of atomic behaviors.

The data is collected by gluing a tiny gold wire (about 1/100th the thickness of a human hair) to the insect and recording changes in electrical properties as it feeds on a plant. In Fig. 3, we show a section of normal insect behavior that ends with a very obvious anomaly. A laboratory door was inadvertently opened, and the resulting breeze lifted the tethered insect off the plant.

While the Matrix Profile [33, 54] and Telemanom (a highly cited SOTA deep learning approach) [21] are unable to detect this anomaly, our proposed feature-based method, $C^{22}MP$, strongly peaks the moment it encounters the anomaly. As we will show in Sect. 6, there are many other domains/situations for which a feature-based approach is more suitable. It is important to note that we are not proposing feature-based methods could replace shape-based approaches, instead, we are suggesting that they can complement them.

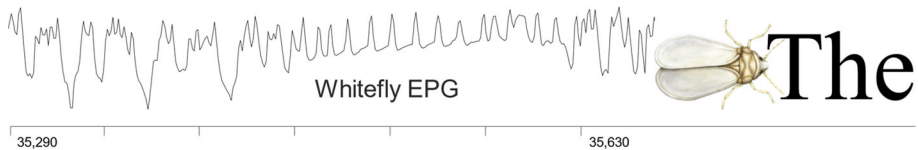


Fig. 2 *Left* A snippet of the data used in the experiments in Fig. 3. *Right* A whitefly (*Bemisia tabaci*) shown approximately to scale with the font used in the main body of this paper

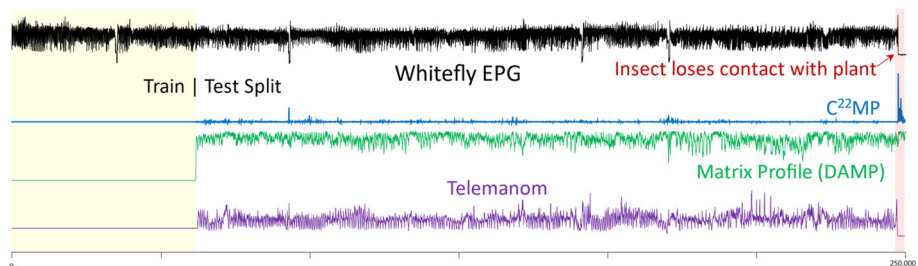


Fig. 3 *Top* Forty minutes of normal insect behavior that ends with a visually obvious anomaly. *Bottom* Three TSAD algorithms, including the one proposed in this work, evaluated on this dataset. Only $C^{22}MP$ correctly peaks at the right place (extreme right)

On a side note, it is worth providing a preview of some timing results here. Our proposed C²²MP took a total of 5.5 min, about 7.2 times faster than real time (Telemanom took a total of 14.6 min). This scalability is important, Wu & Keogh have argued that progress in TSAD algorithms has been hampered by the community's insistence on confining their attention to tiny datasets¹ [49].

3 Background and related work

3.1 The problem we are solving

Prior to discussing related work, we wish to clarify exactly *what* problem we are attempting to solve here. A complete anomaly detection system can be described as consisting of two components:

1. Point to the K locations that are most likely to contain a subsequence that will be considered anomalous.
2. Make K decisions on whether to sound an alarm at the locations pointed to.

In certain TSAD systems, these two tasks are inseparably entwined. Nevertheless, we strongly believe that they should be completely divorced. The reason is that task '2' can potentially avail of additional (possibly out-of-band) knowledge and context. For example, in some manufacturing scenarios, it is common to get false alarms at 12 am, 8 am and 4 pm. These are caused by the minor disruptions of operator shift changes. If a user knows this, she can have a higher threshold for anomalies around these times. Moreover, the binary nature of task '2' is often a false dichotomy. Instead of triggering a single possible action, i.e. (red-alert), in practice we may want a hierarchical response (beep, log, alert_{manager}, alert_{engineer}, ..., red-alert). Finally, echoing the caution expressed by Wu and Keogh [49] and other recent papers, if we are not careful evaluating the results of task '2' only can yield a highly deceptive notion of our actual performance.

With that context, here we are solving task '1', pointing to the K locations most likely to contain an anomaly. Note that this ability can be used by other downstream algorithms such as attention focusing and summarization.

3.2 Related work

Anomaly detection methods vary depending on the type of data being analyzed. In the case of non-temporal data, such as spatial data, anomalies can be detected by measuring the deviation of abnormal points from the rest of the data or by clustering the dataset and identifying points in less dense regions as anomalies. Spatial data is generally assumed to consist of independent data points. On the other hand, time series data is different because the data points are not completely independent, at a minimum, the latest data points in the sequence influence their following timestamps. Consider the following example to understand the limitations of point-wise analysis in detecting anomalies. Let us say we have a time series that records the temperature of an equipment, with values as follows: 40, 44, 41, 44, 42, 43, 44, 44, 95, 97, 96, 97, 98, 98, 96, 99. If we analyze these values independently, most anomaly detection methods would fail to detect the significant step change from 44 to 95 in the data. Instead,

¹ The two most cited datasets for evaluating TSAD algorithms are tiny: NY-Taxi (length 10,320) and Yahoo! Webscope (mean length 1415) [49].

they would simply detect two equally distributed clusters. However, by incorporating the temporal dependencies of samples in a time series, anomaly detection models can capture the context and *shape* of regular patterns in the data and identify any anomalies. As a result, many techniques define anomalies as the subsequences that are maximally far from their nearest neighbor(s) or the time series *discord* [33, 51, 54].

For instance, in [33] the Discord Aware Matrix Profile (DAMP) is introduced. This fast throughput algorithm (exceeding 300,000 Hz) is simple and only requires a single parameter to be set or learned. Unlike the classic Matrix Profile [54], DAMP is not confused by repeated anomalies (twin-freaks) and can handle concept drift in time series. However, this algorithm is limited to detecting shape-based anomalies only.

There are several other machine learning methods [23, 36, 50], which try to detect shape-based anomalies using clustering methods on time series subsequence. For instance, in [36], *K*-Means is used to cluster subsequences of time series extracted by a moving window. In these methods to detect anomalies, the desired number of clusters, *K*, is defined, and the *K*-Means algorithm is executed until convergence, producing *K* centroids. The distance between each subsequence and its nearest centroid is then computed to identify anomalies. Any subsequence that exceeds a predefined threshold from its nearest centroid is classified as an anomaly. In their work, Keogh and Lin [26] have demonstrated that using *all* subsequences of time series data for clustering algorithms is meaningless. They found that the cluster centers obtained through multiple runs of the *K*-means algorithm on the same dataset are no more similar to each other than those of a random walk dataset. This means that the resulting centroids of a dataset could be those of a random walk, and it would be impossible to differentiate between them. Keogh and Lin also tried other algorithms such as hierarchical clustering, but they received the same results. They tested various distance measures, including Manhattan, *L*-infinity, and Mahalanobis distance, and applied *K*-means with $K = 3$ and $w = 128$ on the CBF dataset. They found that the resulting centroids were sine waves that were completely different from the instances in the CBF dataset. Many authors have attempted to analyze this behavior mathematically, and others have tried to solve this problem, or at least show time series patterns that would work with Subsequence time series Clustering (STSC). However, these problems generally remain unsolved.

In contrast to such *shape*-based methods, deep learning approaches typically “*model complex nonlinear feature interactions*” [1, 21, 38]. Although these models can identify feature-based anomalies, the features themselves are generally opaque to the user, and overfitting remains a significant challenge. Additionally, these models involve a substantial number of hyperparameters that must be managed, including:

- The number of hidden layers in the network (depth of the network)
- The number of nodes in each layer (width of the network)
- The length of the window, w
- The learning rate
- The optimization function

If Convolutional Neural Networks are used, another set of parameters must also be handled, such as:

- The architecture of the CNN, such as using Batch Normalization, Dropout, or Max Pooling layers
- The number of kernels in each convolution layer
- The size of the kernel
- The depth of the Convolution Layer

A recent paper [18], explicitly lists the eighteen parameters they needed to set to solve various time series anomaly detection tasks (Learning Rate, Dropout Rate, Dim Feedforward, Batch Size, Encoder Layers, Decoder Layers, Activation Func, Time Warping, Time Masking, Gaussian Noise, Linear Embedding, Phase Type, Self-Conditioning, Layer Norm, Pos. Enc. Type, FFN Layers, Window Size [18]). This alone might give us pause, but Wu and Keogh [49] had previously shown that many of the datasets they considered, including SWaT (c.f. Figure 12), SMAP, MSL and NAB can actually be solved with a single line of code. This mismatch between the complexity of the algorithm and the simplicity of dataset/problem seems like a recipe for overfitting, especially when the number of samples in the training data is often limited, we adopt a more straightforward and parameter-lite approach to detect feature-based anomalies.

Our work exploits the success of the catch22 feature set for time series [1, 33]. This set of features was discovered through a brute force search through thousands of candidate features, averaged over hundreds of datasets from diverse domains, using classification accuracy as an objective function. These features are designed to be a complete and (largely) independent universal feature set. Since its introduction there has been a flurry of activity in using catch22 for tasks such as time series classification and clustering. However, to the best of our knowledge, there are only a handful of efforts to exploit catch22 for anomaly detection [1]. In every case, they simply use catch22 features as inputs to deep learning algorithms. However, this means that we inherit the complexity, opaqueness, and sloth of deep learning algorithms. In contrast, we propose to directly use the catch22 features to produce a simple, transparent and fast algorithm.

The field of TSAD (Time Series Anomaly Detection) is highly active [21, 33, 49, 54], with numerous methods proposed [21, 44, 45, 47]. However, reviewing all of these different techniques is beyond the scope of this paper. Interested readers are directed to [49] and its references for a more comprehensive review.

4 Definitions and notations

In this section, we introduce all the necessary notations and definitions to frame our contributions. We begin by defining the key terms used in this work. The data we are addressing is *time series*.

Definition 1 A *time series* T is a sequence of real-valued numbers t_i : $T = [t_1, t_2, \dots, t_n]$ where n is the length of T .

Figure 4 demonstrates this notation on a toy dataset we use as a running example. This dataset has a local distortion (in this case, *noise*) which *may* be an anomaly, depending on the context.

We are interested in local *subsequences* of the times series.

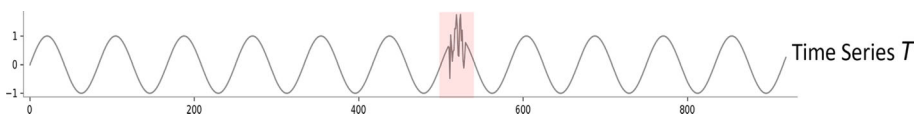


Fig. 4 Our running example has a local burst of noise, which *may* be an anomaly, depending on the context

Definition 2 A subsequence $T_{i,m}$ of a time series T is a continuous subset of data points from T of length m starting at position i . $T_{i,m} = [t_i, t_{i+1}, \dots, t_{i+m-1}]$, $1 \leq i \leq n-m+1$.

The length of the subsequence is typically set by the user based on domain knowledge. For example, for human or animal activities there is a cycle of a daily pattern so a subsequence of 24 h (the circadian rhythm) might be appropriate.

In our proposed algorithm, we need to calculate *all-pairs-similarity* of *all* subsequences of a given time series. The notion of all-subsequences set is purely for notational purposes. In our implementation, we do not actually extract the subsequences in this form as it would require significant time and space overhead. We define an *all-subsequences set* of a given time series as a set that contains all possible subsequences from the time series:

Definition 3 An *all-subsequences set* A of a time series T is an ordered set of all possible subsequences of T obtained by sliding a window of length m across T : $A = \{T_{1,m}, T_{2,m}, \dots, T_{n-m+1,m}\}$, where m is the user defined subsequence length. We use $A[i]$ to denote $T_{i,m}$.

We consider two subsequences to be similar if they have similar features; more specifically if they have similar *catch22 features*. We calculate the catch22 features of each subsequence in the all-subsequences set and generate the *feature profiles*:

Definition 4 The *feature profiles* $FP \in \mathbb{R}^{(n-m+1) \times 22}$ is an array containing the catch22 features of all the subsequences in an all-subsequences set. We use $FP[i, :]$ to denote the subsequence $T_{i,m}$'s features. Each column in the FP array corresponds to one of the twenty-two features' profile.

Figure 5 illustrates this concept, with each row representing the feature profile corresponding to one of the catch22 features computed with a sliding window of length $m = 80$. Note that:

- Some of the features (i.e., '1' and '2') seem to be invariant to the burst of noise.
- Some features *increase* in the presence of the distortion.
- Other features *decrease* in the presence of the distortion.
- Some features (i.e., '3' and '16') have variability that seems unconnected to the presence of the distortion.

The subset of features that are sensitive or invariant to a given distortion depends both on the type of distortion and the data itself. This observation correctly suggests that the choice of the subset of features to use will be critical for the success of any TSAD algorithm based on C22 features.

If we take any subsequence $T_{i,m}$ as a query, calculate its feature-based distance from all subsequences in the time series T and store the distances in an array in order, we obtain a *distance profile*.

Definition 5 *Distance profile* D_i for time series T refers to an ordered array of feature-based distances between the query subsequence $T_{i,m}$ and all subsequences of T . Formally, $D_i = d_{i,1}, d_{i,2}, \dots, d_{i,n-m+1}$, where $d_{i,j}$ ($1 \leq i, j \leq n-m+1$) is the feature-based distance between $T_{i,m}$ and $T_{j,m}$.

For the distance profile D_i of query $T_{i,m}$, the i^{th} position represents the feature-based distance between the query and itself, so the value must be 0. The values before and after position i are also close to 0, because the corresponding subsequences have overlap with query. Our algorithm neglects these matches of the query and itself, and instead focuses on *non-self-match*.

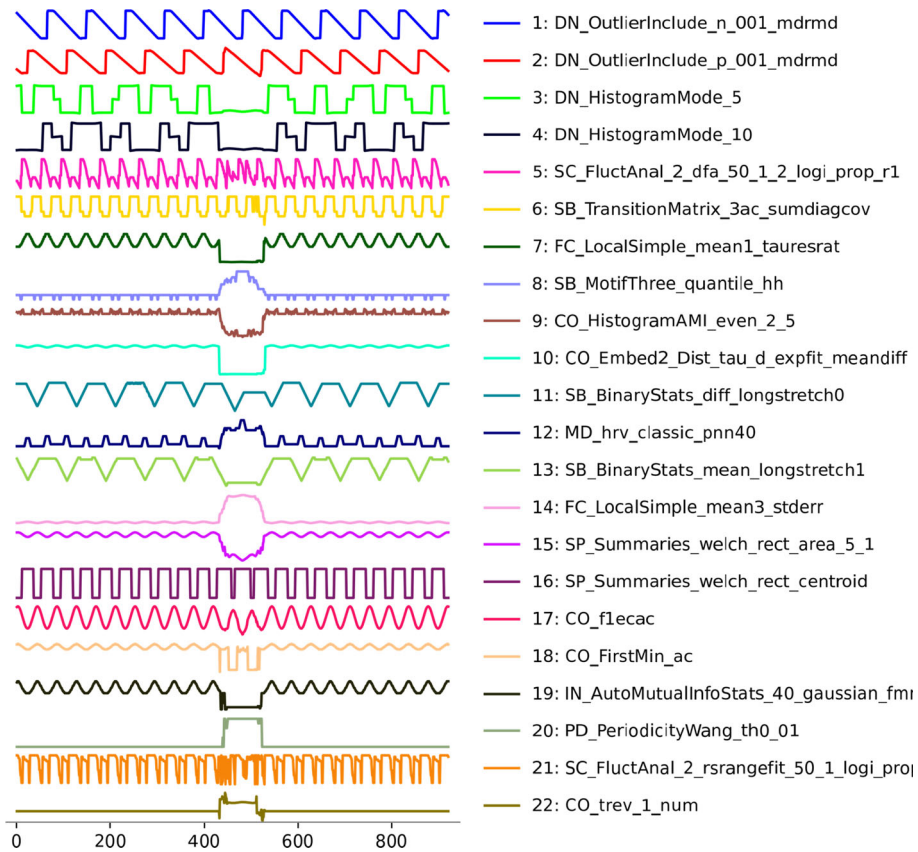


Fig. 5 C22 Feature profiles for the running example introduced in Fig. 4

Definition 6 *Non-self-match*: Given a time series T containing a subsequence $T_{p,m}$ of length m starting at position p and a matching subsequence $T_{q,m}$ starting at q , $T_{p,m}$ is a *non-self-match* to $T_{q,m}$ with distance $d_{p,q}$ if $|p-q| \geq m$. A non-self-matching set M_p of $T_{p,m}$, contains all such non-self-matching subsequences.

While the word *discord* has become something of a synonym for *anomaly* in recent years, for clarity, we reserve the word *discord* for its original meaning [33, 54]. For our proposed variant, the subsequences with the *feature* vector whose distance to its nearest neighbor feature vector is maximum, we use *discordia* (the Latin origin of *discord*). We can now define *time series discordia*.

Definition 7 *Time series discordia*: Given a time series T , the subsequence $T_{r,m}$ of length m beginning at position r is said to be a *discordia* of T if the feature-based distance between $T_{r,m}$ and its nearest non-self-match is maximum. That is, $\forall$ subsequences $T_{c,m}$ of T , non-self-matching set M_r of $T_{r,m}$, and non-self-matching set M_c of $T_{c,m}$, $\min(d_{r,M_r}) > \min(d_{c,M_c})$.

Intuitively the difference between time series discord and time series discordia is that a *discord* is a subsequence that is unique in its *shape*, and a *discordia* is a subsequence that is unique in its *features*.

Exploiting the closeness of the concepts of “discord” and “discordia”, we adopt the idea of Matrix Profile to locate time series discordia. It has been shown that Matrix Profile is one of the most effective ways to find discords [54]. The Matrix Profile (MP) is a data structure that annotates a time series by recording each subsequence’s *Z-normalized Euclidean distance* to its nearest neighbor.

Note that our framing of our contributions as a variant of the Matrix Profile is very deliberate. There is a large and growing body of literature on the MP and a diverse community that uses and extends the MP for diverse tasks [5, 33, 39, 54]. Such users will find our contributions familiar and initiative and should be able to “plug” our algorithm into their frameworks with ease. We introduce the *Catch22 Matrix Profile* ($C^{22}MP$) a data structure that annotates a time series by recording each subsequence’s *feature-based distance* to its nearest neighbor.

Definition 8 A $C^{22}MP$ of a time series T is a vector storing the feature-based distance between each subsequence and its nearest non-self-match. Formally,

$$C^{22}MP = [\min(D_1), \min(D_2), \dots, \min(D_{n-m+1})],$$

where D_i ($1 \leq i \leq n - m + 1$) is the distance profile of query $T_{i,m}$ in time series T . It is easy to see that the highest value of the $C^{22}MP$ is the time series discordia.

As we will explain below, we can compute a special $C^{22}MP$ which only looks to the past. We call it the *left* $C^{22}MP$.

Definition 9 A *left* $C^{22}MP$ of a time series T is a vector that stores the feature-based distance between each subsequence and the nearest non-self-match appearing before that subsequence. Formally, given a query subsequence $T_{i,m}$, let $D_i^L = d_{i,1}, d_{i,2}, \dots, d_{i,i-m+1}$ be a special distance profile that records only the distance between the query subsequence and all subsequences that occur before the query, then we have $C^{22}MP^L = [\min(D_1^L), \min(D_2^L), \dots, \min(D_{n-m+1}^L)]$.

As mentioned earlier, in almost all domains, using *all* twenty-two of the catch22 features will produce poor results. To address this issue, we can use a weight vector that indicates the importance of each feature for the given domain.

Definition 10 A weight vector, w , is a list representing the importance of each feature, where the sum of the total weights is equal to one. Formally, $\sum_{i=1}^{22} w_i = 1$, $w_i \in (0, 1)$.

Note that the weight vector can be real-valued. However, in many cases we may just want to use f equal-weight features, typically with $f \ll 22$. In this case, we simply set the weight of each desired feature to $1/f$.

In Sect. 5.3, we will present several different methods to set these weights, depending on the user’s access to labeled data, domain expertise or ability to obtain feedback from a user.

5 The $C^{22}MP$

With the essential notations and definitions established, we are now in a position to explain how to calculate the $C^{22}MP$, and how to set the feature weights. The $C^{22}MP$ calculates all the pairwise feature-based distances between each possible pair of subsequences. However, for real-time anomaly detection, we only need to look backward and recognize if the incoming data and the subsequence to which that data belongs are anomalous. Therefore, to accelerate the computation, instead of calculating all pairwise distances, we can calculate the *left* $C^{22}MP$ instead.

5.1 The *leftC²²MP*

In Table 1, we outline a brute force algorithm to compute the *leftC²²MP*, assuming the feature weight vector w has already been determined and the user has chosen m . For clarity, we only show the top-1 case. The generalization to the top- K case is trivial.

In line 1, we use the subroutine in Table 2 to obtain the feature profiles of the time series T . We scan all possible subsequences in test data in line 4. For each subsequence, we search to the left for its nearest neighbor. To consider the worst-case scenario, in line 5, we set the distance of the candidate subsequence to its leftmost nearest neighbor to infinity. In lines 6 to 10, we iterate through all the subsequences that are before the candidate subsequence to check which one of them has the lowest distance to the candidate subsequence.

We skip the subsequences in the exclusion zone to avoid trivial matches [54]. Finally, in line 10 we use the distance of the candidate subsequence and its leftmost nearest neighbor (which is saved in *lbsf*) to fill the *left_c22mp* array. Figure 6 demonstrates the output of this algorithm on the running example introduced in Fig. 4.

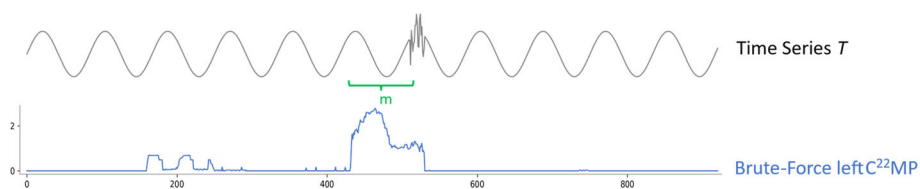
To calculate the feature profiles, we use the subroutine in Table 2 as follows: we iterate through all possible subsequences of length m in T and calculate their catch22 features in lines

Table 1 Brute force computation of *leftC²²MP*

Function: <i>Brute_Force_Left_C22MP</i> (T, m, w, s_idx)	
Input:	
T : Time series	
m : Subsequence length	
w : Feature's importance weights	
s_idx : Location of split point between training and test data	
Output:	
<i>left_c22mp</i> : <i>leftC²²MP</i>	
1	$FP = \text{get_feature_profiles}(T, m, w)$ <i>//get feature profiles (def. 4)</i>
2	$left_c22mp = \text{zeros}((\text{len}(T)-m+1,))$
3	<i>//Scan all subsequences in test data and calculate left_c22mp (def. 9)</i>
4	for i in $\text{range}(s_idx, \text{len}(T) - m + 1)$:
5	$lbsf = \text{inf}$
6	for j in $\text{range}(i-1-\text{exclude_zone}, -1, -1)$:
7	$d = \text{distance}(FP[i], FP[j])$
8	if $d < lbsf$:
9	$lbsf = d$
10	$left_c22mp[i] = lbsf$
11	return $left_c22mp$

Table 2 Computation of *feature profiles*

Function: <code>get_feature_profiles(T, m, w)</code>	
Input:	
T : Time series	
m : Subsequence length	
w : Feature's importance weights	
Output:	
FP : weighted feature profiles	
1	$FP = \text{zeros}((\text{len}(T)-m+1,))$
2	<i>//calculate the catch22 features for each subsequence (def. 4)</i>
3	for i in $\text{range}(\text{len}(T)-m+1)$:
4	$\text{subsequence} = T[i:i+m]$ <i>//get the subsequent (def. 2)</i>
5	<i>//pass w to apply weights while calculating catch22 features (def. 10)</i>
6	$FP[i] = \text{cal_catch22_feature}(\text{subsequence}, w)$
7	$FP = \text{scale}(FP)$ <i>// scale feature profiles</i>
8	return FP

**Fig. 6** Top-1 *leftC*²²MP computed on our running example

3–6. Also, we apply the weights to each feature value, respectively. If a feature has a weight of zero, it means the feature is not used, so to speed up the feature profiles' computation we can omit this feature. Finally, in line 7, we scale the feature profiles to make them unitless and commensurate.

We noted that in Table 2, in line 7 that we scale the feature profiles to make them unitless and commensurate. We use min–max normalization to rescale the range of all the features in the range of $[0, 1]$. Note with this scaling for offline analysis, we can guarantee that the scaled sample will be in the desired range, however, that is not true for *online* prediction. It is possible that new incoming data will be outside the bounds of the minimum and maximum values; therefore, the new scaled data will exceed the range of 0 and 1. Consequently, when using ORR in an online setting, we need to check for these observations prior to making predictions and either remove them from the dataset or limit them to the predefined maximum or minimum values. In practice, we do the former.

5.2 Fast computation of *leftC*²²MP

The brute force algorithm shown in Table 1 correctly computes the *leftC*²²MP but is too slow to be practical for large datasets. Here, we will show how to accelerate its computation. The key insight is that, to extract the top- K discordia, we only need to have exact values for the K largest values in the *leftC*²²MP. For all other values, it suffices to know any upper bound that is less than the top- K value.

For example, suppose we are computing the discordia value for the j th subsequence, and further suppose, that for a previously seen subsequence at i , we have recorded the *best-so-far* discordia value of 4.5. As we begin to compute the value of the j th subsequence, we must start by initializing a variable to infinity, and then we need to update its value every time we encounter a closer match than previously encountered. Thus, the value of the variable will monotonically decrease: $\infty, 7.2, 5.3, 4.2, 2.9, 2.6, 2.2, \dots$ However, once we encountered the value of 4.2, we could have admissibly abandoned our search, as its value is less than our current *best-so-far* of 4.5.

The utility of this early abandoning idea depends on how quickly we find any neighbor that is closer than our *best-so-far*. In the best case, it could be after examining a *single* neighbor, but in the worst case it could require a search through all the data. Nearest neighbor searches can often be accelerated by indexing; however, it would be essentially impossible to build an index in real time on a fast moving stream. However, there is a simple strategy we can use to accelerate our early-abandoning search. Instead of a search *forward* from *time zero* to *now* (present time), we can search *backward* from *now* to *time zero*. The reason why this can be expected to be more efficient is simply that there is generally significant autocorrelation in the feature profiles, and much less correlation between current feature vectors and feature vectors in the distant past. This is because almost all systems slowly drift over time. This simply means that all things being equal, the more recent data will be more likely to be similar to current data.

In Table 3, we formalize these observations with the ORR (Observed Repudiation of Regularity) algorithm.

In line 1, we obtain the feature profiles of the given time series T . In lines 5–17, we iterate through all possible subsequences in test data and find their approximate left nearest neighbor as follows: for each subsequence (skipping the exclusion zone, as per Definition 6), we go back till we find a neighbor that has a distance less than *best-so-far*. We use that distance to fill the a_left_c22mp . If we go back so far that we reach the beginning, this means that we have a new best discordia. In that case, we not only use the nearest neighbor distance to fill the a_left_c22mp but we also use it to update the *best-so-far*.

Figure 7 provides visual evidence of the correctness of ORR.

Note that while the output of the brute force algorithm and our swifter ORR algorithm can diverge, the following properties are true:

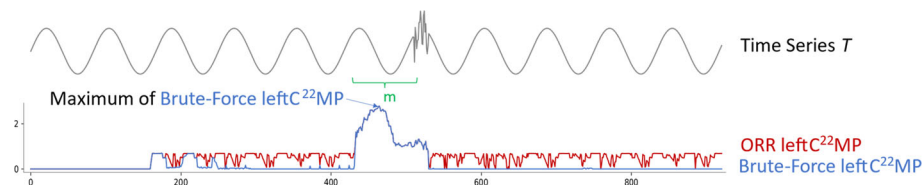
- The two outputs *exactly* agree at the location of the maximum of the brute force algorithm.
- No value in *leftC*²²MP is greater than the maximum of the brute force algorithm.

These properties combined mean that, just as with the brute force algorithm, we can report the K highest values of the output of the ORR algorithm as the top- K anomalous regions.

As shown in Sect. 6.10, the early-abandoning algorithm is several orders of magnitude faster than brute force search and allows us to address datasets with billions of datapoints.

Table 3 ORR: early-abandoning computation of $\text{leftC}^{22}\text{MP}$

Function: $\text{OOR}(T, m, w, s_idx)$	
Input:	
T : Time series	
m : Subsequence length	
w : Feature's importance weights	
s_idx : Location of split point between training and test data	
Output:	
a_left_c22mp : approximate $\text{leftC}^{22}\text{MP}$	
1	$FP = \text{get_feature_profiles}(T, m, w)$ <i>//get feature profiles (def. 4)</i>
2	$\text{left_c22mp} = \text{zeros}((\text{len}(T)-m+1,))$
3	$bsf = 0$ <i>// The current best discordia score</i>
4	<i>//Scan all subsequences in test data and calculate approximate left_c22mp</i>
5	for i in $\text{range}(s_idx, \text{len}(T) - m + 1)$:
6	$lbsf = \text{inf}$
7	for j in $\text{range}(i-1-\text{exclude_zone}, -1, -1)$:
8	$d = \text{distance}(FP[i], FP[j])$
9	if $d < lbsf$:
10	$lbsf = d$
11	if $d < bsf$ and $j > 0$:
12	Break
13	elif $d \geq bsf$ and $j > 0$:
14	continue
15	elif $d \geq bsf$ and $j == 0$:
16	$bsf = lbsf$
17	$a_left_c22mp[i] = lbsf$
18	return a_left_c22mp

**Fig. 7** A comparison of the output of the top-1 brute force and the ORR algorithm on a small toy dataset

5.3 Setting the feature weights for C²²MP

The ORR algorithm introduced in Table 3 allows the efficient discovery of *discordia*, but if we use all twenty-two of the C22 features we may be doomed to poor performance. In particular, we may be condemned to reporting many false positives if we use features that are irrelevant to the anomaly detection task. To see this, we can revisit the example shown in Fig. 1. To enhance the flow of the introduction, we originally presented the better clustering simply as catch22. However, as made clear in Fig. 8 *right*, this clustering was obtained by using a *subset* of catch22 features. Had we used all the features, we would have obtained the clustering shown in Fig. 8 *left*. Note that this clustering is still better than using Euclidean distance, but it becomes evident that a judicious selection of features can greatly benefit catch22.

While this suggests that a good choice of features weights will improve the sensitivity and specificity of the *left*C²²MP, we think it is very unlikely that there exists a single best strategy to find such weights. An anomaly detection algorithm can be deployed in diverse settings, spanning the space of no-labeled-data vs. copious-labeled-data, no-domain-knowledge vs. rich-domain-knowledge, etc. Instead, in this section, we will consider a handful of techniques to discover the appropriate weights for the C22 features.

5.3.1 Hand coded subsets of features

In certain instances, practitioners can leverage their domain expertise to manually design a suitable subset of features. For example, suppose that we know that for our domain it is possible that the typical patterns might be observed flipped upside down, and that this is *not* to be considered anomalous. However, if the patterns are ever observed flipped left-to-right, that is indicative of an anomaly.

Knowing this, we can use features that are not sensitive to data appearing flipped upside down. These include features {5, 6, 7, 8, 9}. However, we want to avoid using features that are sensitive data appearing flipped upside down, these include features {1, 2, 5, 21}. In order to help such practitioners, we have created a detailed key of all the invariances of catch22

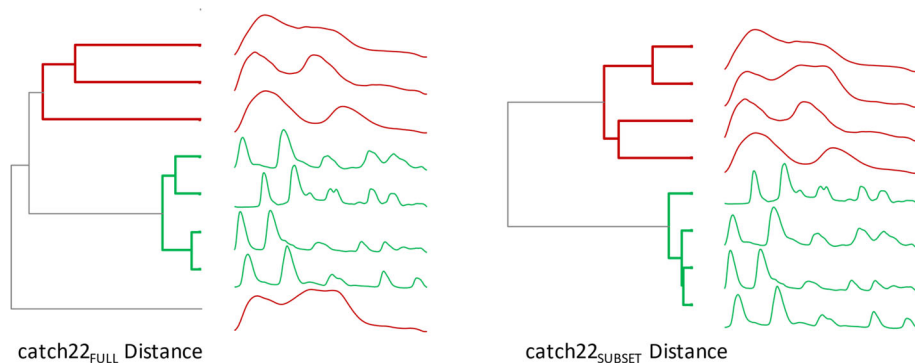


Fig. 8 Instances from the 50words dataset clustered using the full set of catch22 features (*left*) and a subset of catch22 features (*right*). The full set produces a reasonable clustering, but incorrectly considers a red object to be an outlier (cf. Figure 1)

Table 4 Pseudocode for feature search with labeled data

Function: feature_search(T, y)	
Input:	
T : a set of normal and anomaly time series	
y : labels for each sample	
Output:	
w : Feature's importance weights	
1	$transformed_T = []$
2	for t in T :
3	$transformed_T.append(cal_catch22_features(t))$
4	$model = Classifier()$
5	$model.fit(transformed_T, y)$
6	$w = model.feature_importances$
7	return FP

[10].² This is an attractive solution for users that have both a familiarity with c22 features and a strong initiative understanding of their domain of interest. Since such users may be rare, below we consider other possibilities.

5.3.2 Feature search with copious training data

In a handful of situations, we may have copious training data to learn an appropriate feature weighting. Here, we must ward off a possible confusion. There are papers in the literature that claim to be doing *anomaly detection*, but are arguably doing *classification*, with “anomaly” simply being the rarer class. One could critique such efforts by noting that if we have examples of the anomaly in advance, this is simply classification or pattern matching. The normal definition of anomaly suggests an unknown/unexpected pattern, precluding the possibility we have examples in our training data.

However, here we are not suggesting we learn the anomaly *patterns*, we are simply learning which *features* are sensitive to disturbances in the domain of interest. We propose to learn these features by treating the training data as a classification problem and optimizing the classification error-rate by a simple greedy forward search. Table 4 demonstrates the pseudocode for this idea.

In lines 1–3, we iterate through all samples in the labeled data and calculate the catch22 features for each time series. We create a classification model in line 4, then train this model to classify normal and anomaly samples in line 5. Finally, we extract the feature importance learned through classification in line 6. Note we can use any explainable classification model in this method. For example, we can use a Decision Tree Classifier, in which the feature

² This contrived example is not as implausible as it may seem. Suppose we are monitoring the accelerometer time series from a smartphone in a user's pocket. If the user takes a call, and then returns the phone to her pocket upside down, the Y-axis time series will flip *upside down*, but will not be *flipped backwards*.

importance scores are estimated by calculating Gini gain (the amount of Gini impurity that was eliminated at each branch of the decision tree) or a logistic regression model in which the feature importance scores are estimated by using the learned coefficient value for each feature [43].

We can also use the leave-one-out mechanism with any classification model, where in each iteration we use one of the twenty-two features to classify the samples. Subsequently, we select the feature set that yields the highest classification accuracy.

5.3.3 Feature search by feature profiles

In scenarios where we have limited labeled anomalies (maybe just one or two anomaly samples), the method presented in Sect. 5.3.2 would not be suitable. In such cases, an alternative approach is to analyze the feature profiles in order to learn the weight of each feature. Intuitively, we expect to see a feature in the feature set if its feature profile “responds” to the anomaly of our interest. For example, in our toy example in Fig. 4, we do not expect to have features 1 and 2 in the feature set, as their feature profiles seem to vary independently of the anomaly. This suggests that we can use regression analysis to learn the feature set. We create a regression model to estimate the relationships between the time series (dependent variable) and feature profiles (independent variables); then, we extract the learned relationships (the coefficients). The coefficients can be used as the feature importance score. In general, feature importance refers to how useful a feature is at predicting the target variable, which, for our case, is the time series and the anomaly within it. Table 5 formalizes this idea.

In line 1, we create a linear regression model. To learn the relationship between the time series and its feature profiles, we fit the model to the time series and its feature profiles in line 2. Finally, we extract the coefficient learned during regression as feature importance in line 3. Similar to using a classification model to learn feature’s weights, we can use any explainable regression model in this approach such as linear regression or a Decision Tree Regressor [32].

Table 5 Pseudocode for feature search with feature profiles

Function: <code>feature_search(<i>T</i>, <i>FP</i>)</code>	
Input:	
<i>T</i> : Time series	
<i>FP</i> : feature profiles	
Output:	
<i>w</i> : Feature’s importance weights	
1	<code>model = Regression()</code>
2	<code>model.fit(<i>FP</i>, <i>T</i>)</code>
3	<code>w = abs(model.coefficients)</code>
4	return <i>w</i>

5.3.4 Human in the loop feature search

In Sect. 5.3.1, we proposed to use domain expert's knowledge to choose features. In such cases, we assume that the user has detailed domain knowledge that will help them choose the right features. For example, in the Oil&Gas production domain, conservation laws (i.e., conservation of mass, energy) constrain the values that can be seen in normal telemetry, and features can be chosen to be sensitive to deviations from these [3]. In this section, we propose to exploit less formal and explicit human knowledge. Our only assumption is that a user could recognize an anomaly if she saw one. Given that assumption, could we generate artificial, but plausible anomalies?

If so, our difficulties are over, we could simply revert to one of the feature learning algorithms in Sects. 5.3.2 and 5.3.3.

Here, we are inspired by an observation by Sir D'Arcy Thompson. Figure 9 shows some images from his 1917 book *On Growth and Form* [44], which shows it is possible to reproduce almost all the morphological diversity of fishes, with just a few samples and a small set of geometric transformations.

We believe that we can similarly create plausible synthetic anomalies [29, 46]. Consider the two datasets shown in Fig. 10.

Assuming that we are seeing normal data, how should we weigh features that are sensitive to time warping of the data? This is a difficult question, even if we know which features are sensitive to such distortions (recall we have built a dictionary to help build such intuitions [10]). Moreover, the question depends on the user's task and domain knowledge, which may be intuitive and hard to elicit and represent. However, consider Fig. 11, where we added the same amount of warping to the middle section of each time series.

For the Melbourne Pedestrian dataset, there are a lot of variabilities, but it is mostly manifest in local changes in amplitude (especially weekends vs. weekdays). Adding this amount of time warping creates a visually obvious anomaly. In contrast, for the Gait dataset, the added warping is subsumed within the existing natural variability.

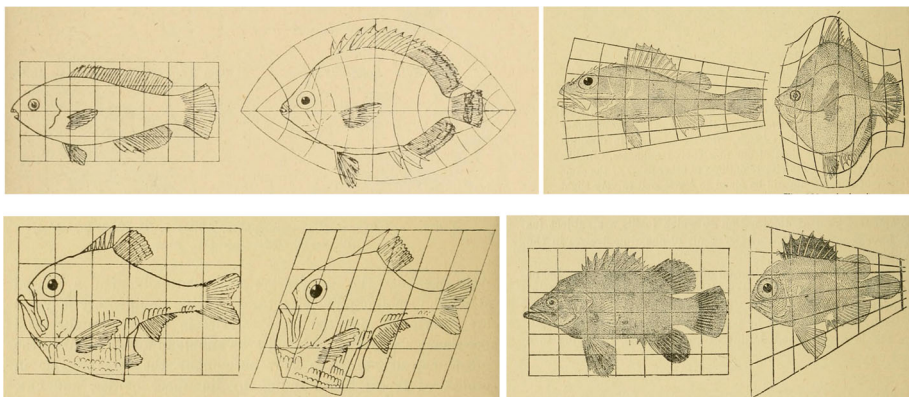


Fig. 9 A sequence of figures from [44] suggests that we can reproduce almost all the diversity of fishes, using just a handful of “prototype” fish, and small number of linear (bottom row) and nonlinear (top row) distortions. For example, the figure in bottom left shows that we can take a known fish, here the Olfer's Hatchetfish (*left*), and apply a simple geometric transformation to obtain another fish that really exists, here the Diaphanous Hatchetfish (*right*)

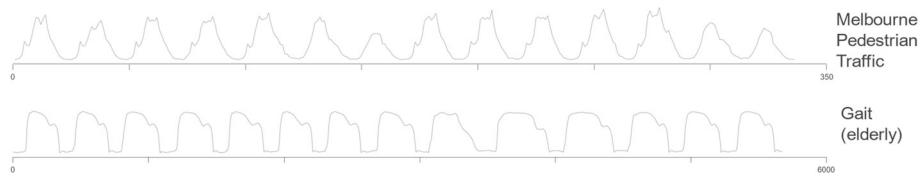


Fig. 10 Two datasets with about 14 cycles shown. Both are periodic, but Melbourne has almost no temporal variability, the rigidity of the 9-to-5 business cycles acts to keep the days in synch. In contrast, the gait cycle shows the natural variability of walking in an indoor environment with obstacles, doorways, changes of direction etc.

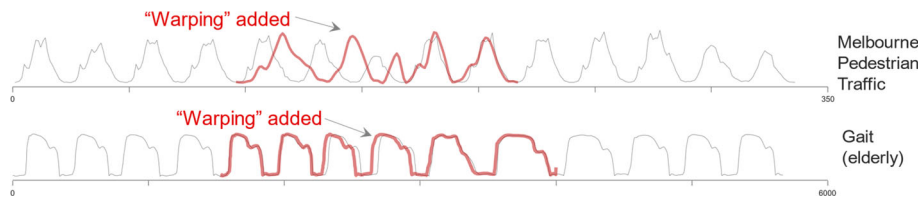


Fig. 11 The two datasets shown in Fig. 10 with the same amount of warping added to each's middle section. The addition of warping looks natural on the gait data but is visually jarring for the Melbourne pedestrian dataset. This domain induced distinction offers a clue as to how we can learn the best set of features for each domain

This example exemplifies our human-in-the-loop approach. For each distortion of interest (including *noise*, *spike*, *warping*, *linear scaling* etc. [9, 10, 48, 52]), we show the user four randomly chosen snippets from their dataset. The plot has an interactive slider that allows the user to increase or decrease the amount of distortion. The user is invited to move the slider to indicate the maximum amount of distortion she would accept before declaring at least one of the four snippets an anomaly.

Once the user has annotated the data in this way, we can simply avail of the feature search algorithm discussed in the previous section to learn the appropriate features. There are several research efforts on time series generation, almost all of which use deep learning [2, 53]. However, we found that for each distortion we could make high-quality examples with just a few lines of code. In [10], we show a video of a typical interaction with a user. Note that the entire process only takes a few minutes.

6 Empirical evaluation

To make certain that our experiments are reproducible, we have built a website [10] that contains all the data/code used in this work. All experiments were conducted on an Intel[®] Core i7-9700 CPU at 3.00 GHz with 32 GB of main memory, unless otherwise stated.

6.1 How should we evaluate TSAD algorithms?

A recent series of papers from various research groups have cast significant doubt on both the common TSAD benchmarks and evaluation metrics [49]. While we have neither the space

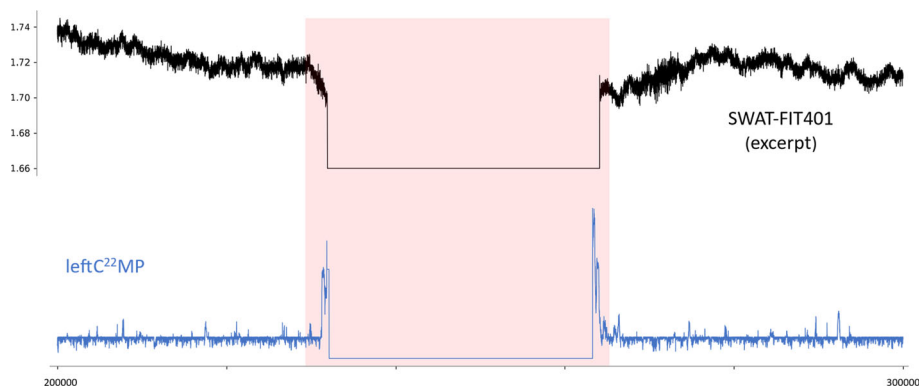


Fig. 12 An excerpt from the SWAT-FIT401 dataset. The ground truth anomaly is highlighted in red. Unsurprisingly $\text{leftC}^{22}\text{MP}$ can easily find this anomaly. The constant region that forms the anomaly actually has a value of exactly zero, we added a constant to it to make the variability of the normal data clearer

nor the inclination to weigh in on this debate, we need to at least briefly explain some of the issues to justify how we do *our* evaluation.

Consider Fig. 12, which shows an excerpt of one dimension (FIT401) of the 51-dimensional SWAT benchmark [15]. This is one of the most cited and used benchmarks in the literature, appearing in at least 200 papers [6, 12, 28, 30]. We ran the fixed weight $\text{leftC}^{22}\text{MP}$ algorithm on this dataset. Unsurprisingly, it strongly peaks at both the transition from normal data to the anomaly, and from the anomaly to normal data.

This example highlights the problems the community has noted:

1. Many benchmark problems are *much* too simple to warrant claims of a successful algorithm. Note that while we did use $\text{leftC}^{22}\text{MP}$ to find the anomaly, we could have found it with a single line of MATLAB [49]: `isAnomaly = (FIT401==0);`
2. We would argue that here that detecting this anomaly should count as a single *binary* success. However almost all papers reason like this: *The anomaly is marked as 35,800 datapoints long in a dataset of 449,921 datapoints, so we should report detecting this with four significant digits!* In our view, this is misleading spurious precision; it implies precise measurements supported by thousands of *independent* datapoints, not a single “blip”. We think this argument to be forceful, but to our surprise, some have pushed back at it.³ We have found the following analogy to be fruitful (with apologies for the graphic imagery).

Suppose we suspect that John, a 100 kg peace worker, was killed by a landmine. Two forensic anthropologists are sent to investigate. Alice finds a large skull fragment, does a DNA test, and concludes that John is dead. Bob finds a leg and a hand, which in total weight 15 kg, does a DNA test, and concludes that John is 15% dead.

We believe that this story is a perfect analogy for the example in Fig. 12. Once we have seen the value of the time series plunge to zero, this sensor is unambiguously “dead”. Any scoring function that *continues* to reward you for finding additional sections of constant region time series is just like suggesting that Alice would be surer that John is dead is she later found a toe fragment. Likewise, if Bob reweights his fragments with a more sensitive scale and then reports John is 15.49% dead, we would think him quite naïve. With this understanding,

³ In blog forums, private conversations, openreview.net etc.

the dozens of papers that report four significant digits on this datasets do seem somewhat naïve. In [12], the authors summarize the results of twenty-seven approaches to SWAT, that appeared in eight unique papers. Sixteen of the approaches are reported with four significant digits, and the rest with three significant digits. It is hard to reconcile such precision with even a casual inspection of this dataset.

3. Problem ‘2’ above is compounded by poor labeling. The begin–end locations for this anomaly are given as 227,900–263,700. The fact that these are multiples of 100 should tell us that these are the rough guesses by the annotator. In fact, careful visual inspection strongly suggests that these labels have an uncertainty *greater than* ± 100 . This uncertainty would not change the *binary* success we advocate for but would change at least two of the four significant digits that most papers report [12].

In summary, we have sympathy for the claims that both the common benchmarks and the evaluation metrics [49] are flawed.

Given these issues, how should we evaluate? We agree with the claim in [39] that one of the best ways to understand a TSAD’s strengths and limitations is to directly plot the scoring function next to the time series, as we did in Fig. 12 above. In this next section, we will do exactly that, on an ambitious scale. In addition, in subsequent sections, we will use datasets and metrics that are free of the flaws discussed above.

6.2 A visual intuition for *leftC*²²MP

With the caveats in the last section in mind, in Fig. 13, we show the performance of *leftC*²²MP on datasets collected from twenty different papers. Clearly, we do not have the space to discuss, or even cite, all twenty papers. However, in [10] we have a document that gives detailed provenances for all datasets and shows larger figures.

These results strongly hint at the generality and effectiveness of *leftC*²²MP, the top discordia peak at the ground truth locations. Nevertheless, they are informal and anecdotal evidence. Moreover, they only show that *leftC*²²MP is expressive enough to *represent* the anomalies, not that we can learn an appropriate set of feature weights to discover them. In the following sections, we will provide more rigorous evaluation.

6.3 The hexagon ML/UCR dataset

While there are many benchmark datasets proposed in the literature, most of them do not allow comparisons among published results. The problem is that the metric for computing a success can vary greatly from paper to paper, making the published results incommensurate [13, 14, 29, 41, 53]. The HEXagon ML/UCR dataset is different in that the 250 datasets come with a concrete and well-reasoned metric of success. For example, different TSAD algorithms may tend to peak at the beginning, or the middle, or the end of an anomaly. These differences are generally inconsequential but combined with a brittle scoring function could make a good algorithm score poorly. The HEX/UCR scoring function allows a little slop (± 100) in the scoring boundaries to make this issue moot.

In addition, the HEX/UCR datasets are unusual in having detailed provenance, largely freeing them from the flaws noted in [49]. Many of the datasets are completely natural, the remainder are anomaly-free datasets that have had exactly one anomaly seamlessly added in a domain-specific way. The datasets come from diverse domains, including medicine, industry, biology and meteorology. Table 6 shows a comparison of our proposed method with rival

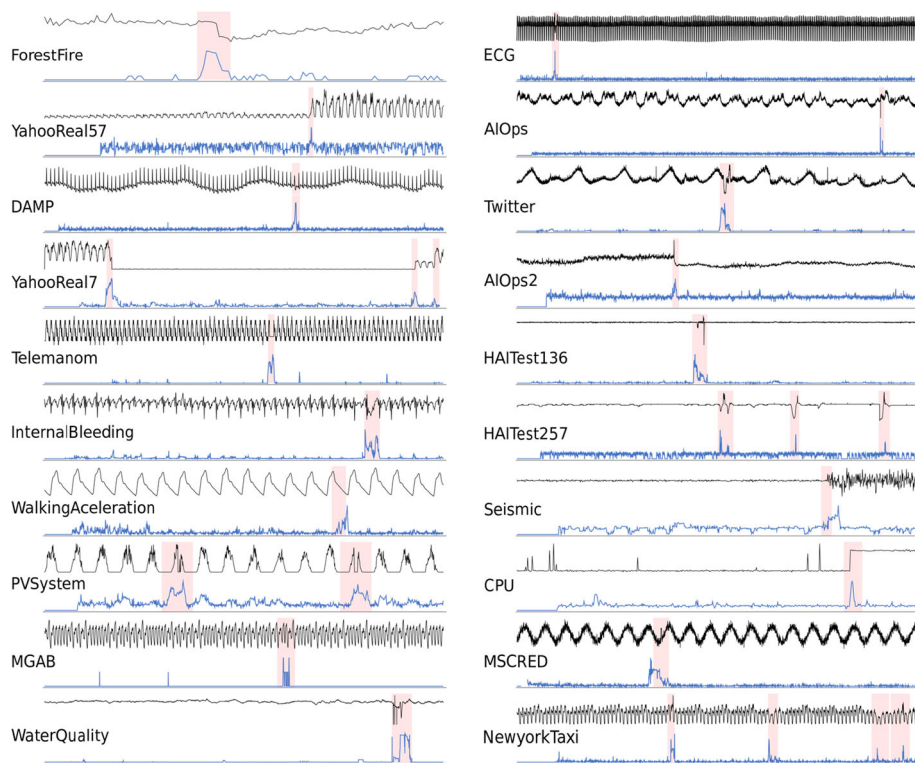


Fig. 13 The performance of $leftC^{22}MP$ on datasets that appeared in twenty different papers in the last two decades

Table 6 A comparison of 14 algorithms on the HEX/UCR dataset

Method	Score	Method	Score
$leftC^{22}MP$	0.568	{{ { DAMP + $leftC^{22}MP$ } }}	{0.692}
DAMP ($leftMP$) [33]	0.556	USAD [4]	0.276
AE [4]	0.236	Telemanom [21]	0.468 ^a
LSTM-VAE [40]	0.198	SCRIMP (<i>full MP</i>) [54]	0.416
RRCF [42]	0.030	MERLIN (generalized MP) [42]	0.440
MDI [42]	0.470	NORMA [8]	0.474
TranAD [42]	0.190	GANF [42]	0.240

^aTelemanom runs out of memory on the larger datasets. This score is extrapolated from the shorter/easier datasets, thus optimistic

approaches. The results highlighted in blue are results that we copied from other works. All other results we computed ourselves.

Our proposed algorithm outperforms all rival approaches. It is slightly better than DAMP, which is regarded as SOTA [33]. The approach labeled {DAMP + $leftC^{22}MP$ } is *not* a true algorithm. It is a post-hoc ensemble where we pick the better of the two algorithms *after* seeing

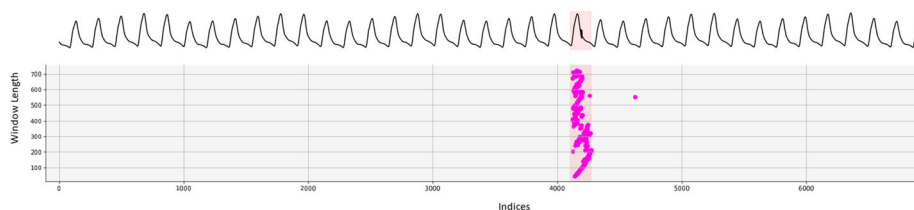


Fig. 14 The m value suggested by DAMP for InternalBleeding dataset is 181. Therefore the range of the m would be from 45 to 723. C²²MP is able to detect the correct location of the anomaly for almost all window length in this range

the labels. The ensemble's exceptionally high score tells us that the two algorithms in question have largely uncorrelated errors and suggests a future research direction in combining TSAD algorithms.

While the format of these datasets and the scoring function allows comparison of *accuracy* across papers, it does not allow comparison of *timing results*. However, even if we assume that all the datasets are sampled at 250 Hz, C²²MP can comfortably process all datasets at least an order of magnitude faster than real time.

6.4 Sensitivity to window length

In the previous section, we showed that *leftC22MP* can score very well on the HEX/UCR datasets, if we use the simple heuristic suggested by DAMP to set the value of m . Nevertheless, it is natural to ask how sensitive the algorithm is to that parameter.

In Fig. 14, we consider the InternalBleeding dataset (chosen because it is one of the shortest datasets from UCR/HEX, and we can plot it in its entirety), and we test every parameter from $\frac{1}{4}$ to 4 of the DAMP suggested value. This plot suggests that the *leftC22MP* algorithm is not particularly sensitive to the setting of m .

6.5 Why is C²²MP so robust?

The results in the previous section may appear a little surprising; deep learning approaches have been much touted in recent years, yet none seem to do particularly well. We believe dataset shown in Fig. 15. A can cast light on why this was the case.

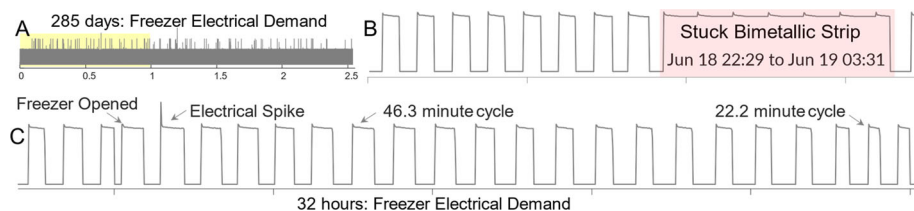


Fig. 15 **A** An electrical demand dataset for a single device, a freezer. **B** The (rather obvious) anomaly caused by a stuck temperature sensor. **C** A random section of the training data

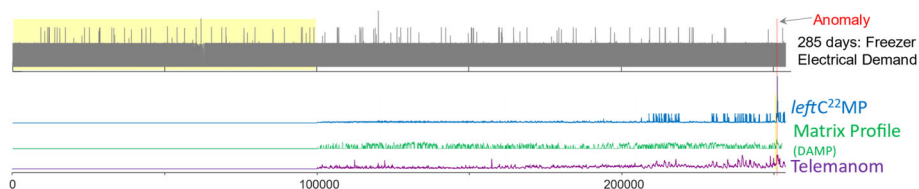


Fig. 16 The $leftC^{22}$ MP can easily find the Stuck-Bimetallic-Strip anomaly. DAMP *just* finds it, and Telemanom fails

In Fig. 15B, we zoom-in to the sole anomaly in this dataset. It does not appear to be particularly challenging, yet all deep learning approaches we tried fail to discover it. An examination of the training, a snippet is shown in Fig. 15C tells us why this is the case.

- The period of the data (the compressor on/off cycles) can vary based on the room temperature, which varies both daily and seasonally. Many published algorithms seem to work well with *strict* periodicity (like the Melbourne dataset shown in Fig. 10) but cannot handle *variable* periodicity.
- The dataset (both the train and test) is replete with variations that are inconsequential. For example, very short off-cycles caused by the freezer being opened, or electrical spikes. These two simple events can appear in arbitrary arrangements that all look essentially the same in the *feature* space, but very different in a *shape* space.
- The training data is 100,000, not a large dataset by modern standards, but this size strains many deep learning approaches.

In Fig. 16, we compare three approaches on this problem. As the figure shows, the $leftC^{22}$ MP easily finds this anomaly but Telemanom dramatically fails. As Telemanom is nondeterministic, we ran it five times; the *best* run had 32 false positives. Moreover, the total time required for a single run is 3.04 h, which contrasts poorly with the 7.25 min required for $leftC^{22}$ MP. Here DAMP, using the subsequence length selection tool suggest in the original paper [33], did *just* find the anomaly, but it would have failed at a slightly different length. In contrast, $leftC^{22}$ MP is successful even if we double or halve this suggested setting for the subsequence length.

To test the correctness of our explanations of Telemanom's (and more generally, most deep learning approaches) poor performance that we posited above, we built a proxy for the dataset with perfect square-waves and tested algorithms on it as we introduced synthetic spikes, variable periodicity, etc. The results (expounded in [10] for brevity) support our observations and suggest research directions for advocates of deep learning TSAD algorithms.

6.6 Case study on medical data

The catch22 features are very expressive, but not completely so. Several papers have needed to augment the features with some domain-specific feature(s) [1, 31]. Here we give an example of where we had to do this, to create C23, with $C23 = [C22 \cup \text{Max}(T)]$.

We consider the task of anomaly detection in telemetry of CCT, Contraction in Cardiac Tissue (related to, but distinct from the more familiar ECGs). A recent paper [37] introduced a sophisticated data generator that takes real data and introduces into it highly plausible anomalies of amplitude, duration, morphology or timing. These anomalies are very interesting: Some are visually apparent; but some, for example, 'Fast Pulse Decay', are difficult to see even if you know where the anomaly is. Moreover, some anomalies are caused not by the

Table 7 A comparison of four TSAD algorithms on CCT data

Anomaly type	Autoencoder	LSTM	<i>leftC</i> ²² MP	<i>leftC</i> ²³ MP
Lower amplitude	0.55	0.83	0.41	0.90
Higher amplitude	0.71	0.99	0.18	0.95
Missing pulse	0.63	0.91	0.97	0.97
Slow pulse decay	0.67	0.95	1.00	1.00
Fast pulse decay	0.61	0.88	0.98	0.98
Early/anticipated Pulse	0.51	0.77	0.99	0.99
Pause/block	0.61	0.90	1.00	1.00
Average accuracy	0.61	0.89	0.79	0.97

The best performing algorithms are highlighted in bold

addition of a new pattern or disturbance, but by the *omission* of an expected pattern (Pause and Missing Pulse).

In Table 7, we show the results of comparing our approach with two bespoke deep learning algorithms. To be fair to deep learning algorithms (which have many sensitive parameters and can be difficult to reproduce), we copied the results from [37] and used identical settings for our experiments (also averaging over 100 runs).

In order to “stress test” our approach, we did not do any feature selection here, we simply used all 22, (and were stated, all 23) features. These results are very promising. Note that we are beating two domain-informed deep learning approaches that are heavily customized for the task-at-hand, with our simple generic approach.

6.7 Learning a threshold

Recall that in Sect. 3.1 we explained that TSAD algorithms can be seen as having two parts:

1. Identify the location that is most likely to contain a subsequence that will be considered anomalous.
2. Make a decision as to whether to actually sound an alarm.

We further explained why we focus our evaluation on the first part. Nevertheless, for completeness we will show how we can address the second task and make a binary decision flag the subsequence as anomalous or not. For this purpose, we simply need to learn a *threshold* on the training data.

To test the effectiveness of ORR on this second task, we conducted an experiment using the CCT data generator that was introduced in Sect. 6.5. We generated one hundred different datasets using different seeds, each of which consisted of eight columns. The first column contained only normal CCT data, while the remaining columns contained one specific type of anomaly: ‘Slow Pulse Decay’, ‘Fast Pulse Decay’, ‘Early/Anticipate Pulse’, ‘Missing Pulse’, ‘Pause/Block’, ‘Higher Amplitude’, and ‘Lower Amplitude’, respectively. We used the data in the first column (the normal time series) as training data. To learn the threshold, we calculated the *leftC*²³MP of the training data across one hundred trials. Since the training data only contains normal CCT, the maximum value of the *leftC*²³MPs can be used as decision threshold. Any sample with an anomaly score higher than the threshold is considered an anomaly. In this experiment we aimed to stress test the effectiveness of C²³MP and its ability

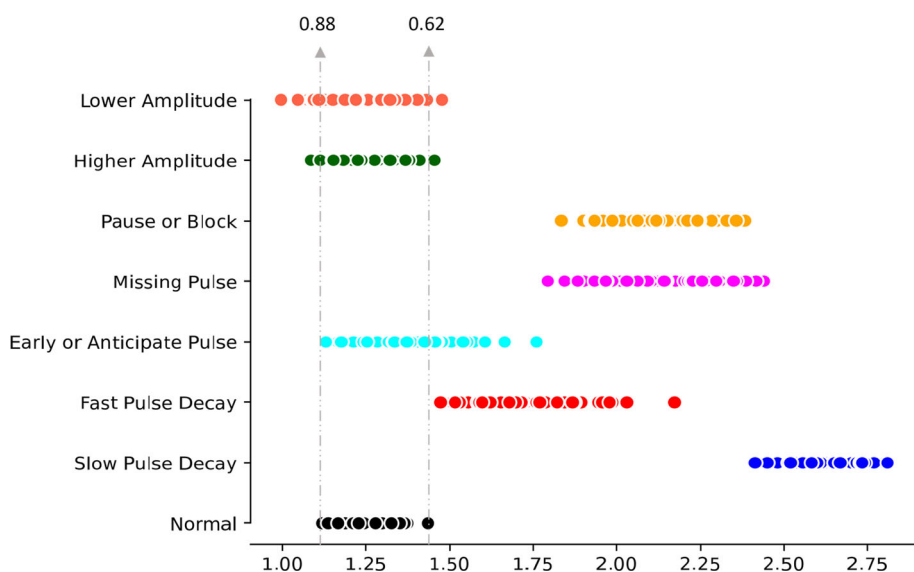
Table 8 The result of $leftC^{23}MP$ on CCT data for the task two

Anomaly type	$leftC^{23}MP$ Threshold = 1.12	$leftC^{23}MP$ Threshold = 1.44
Lower amplitude	0.69	0.02
Higher amplitude	0.83	0.01
Missing pulse	0.99	1.0
Slow pulse decay	1.00	1.0
Fast pulse decay	0.92	1.0
Early/anticipated pulse	0.76	0.38
Pause/block	1.00	1.0
Average accuracy	0.88	0.62

in detecting different type of anomalies. To achieve this, we used the smallest and largest decision thresholds (with an additional epsilon value) obtained from one hundred trials for all the datasets as the threshold. The results of this experiment are presented in Table 8.

Table 8 shows that ORR has an average accuracy of 0.88 when threshold 1.12 is selected. However, the accuracy varies depending on the type of anomaly. Specifically, we can see that ORR performs poorly in detecting ‘Lower Amplitude’, ‘Higher Amplitude’ and ‘Early/Anticipated Pulse’ anomalies, but well in detecting other anomalies. To investigate this further, we analyzed the highest values of $leftC^{23}MP$ s for each type of anomaly across the one hundred trials, as shown in Fig. 17.

We find that different types of anomalies have distinct clusters of high values, and the chosen threshold effectively separates normal from anomalous samples for the cases that we have high accuracy. Interestingly, we observed that for the cases in which ORR performs

**Fig. 17** Comparing the distribution of the highest value across different type of anomalies

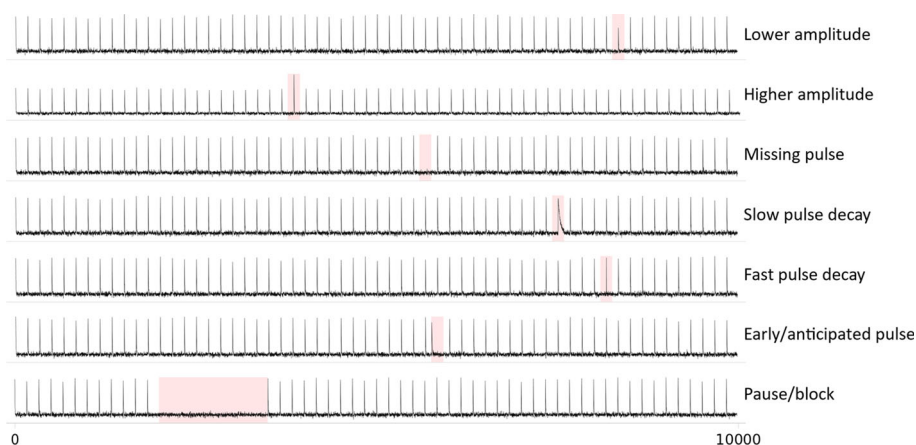


Fig. 18 Examples of the CCT data with different type of anomalies. The cases observed to be difficult for *leftC22MP* in Fig. 17 are also very difficult to detect by visual inspection

poorly, the distribution of high values is very similar to that of the normal case. This suggests that the anomalies in these cases may be subtle, Fig. 18 shows examples of the CCT data with different type of anomalies. As we can see in the ‘Lower Amplitude’ and ‘Higher Amplitude’ only one single sample is distorted and for the ‘Early/Anticipated Pulse’ there is no distortion in the pattern only we have an early pulse.

6.8 Case study on mouse motion capture

Researchers at UCR are building a mouse model to understand factors that influence the onset of Parkinson’s disease. As Fig. 19 shows, one tool used to study the effect of cognitive decline is a treadmill *roller*, and the researchers have many hours of such data.

The biologists are interested in finding two types of anomalies here. First, there are simple feature extraction issues. For example, sometimes the motion capture tool (DeepLabCut [35]) is unable to find the mouse’s paw as it is occluded by its tail (type B in Fig. 19). Such anomalies can then be either manually corrected or excluded from analysis. More interestingly, however, there may be *behavioral* anomalies induced by gene knockouts or medications.

To test our ability to find both types of anomalies, the biologists used visual inspection to provide a short snippet of anomaly-free data, and a much longer region that terminated with an area dense with both quotidian and behavioral anomalies, which they carefully annotated.

After learning the feature weights using the snippet of positive-only data and our human-in-the-loop tool, we discovered the anomalies shown in Fig. 19 *center*. While most anomalies were simple feature extraction issues, two anomalies surprised us. The first anomaly, type D (Fig. 19 *top.right*), depicted a mouse grasping the treadmill and “riding” it around for a complete cycle. The second anomaly, type C, occurred when the mouse quickly “double-tapped” while walking on the roller, this locomotive “stutter” is likely due to a side effect of an induced pathology. This finding is particularly interesting because it is challenging to detect this anomaly with the naked eye. We invite the reader to examine [10] for more details and to view a video of the experiment.

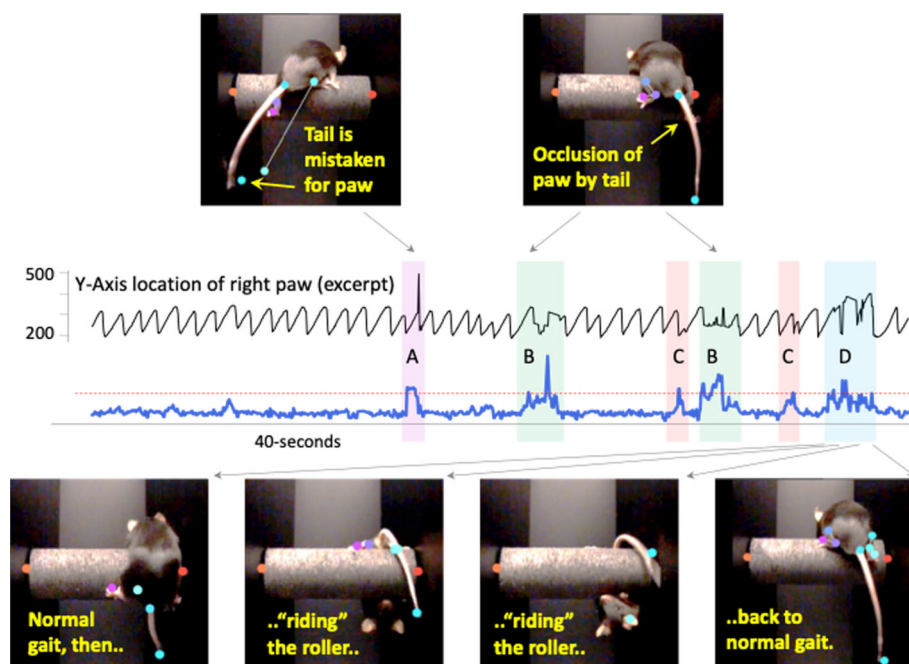


Fig. 19 Center A trace from the right paw of a healthy mouse, with its companion $C^{22}MP$. Top and bottom Screenshots of the video with markers labeled and tracked by the DeepLabCut motion capture tool [35]

6.9 Case study with human in the loop

In this section, we evaluate the utility of using the “human annotation of synthetically distorted data” idea we introduced in Sect. 5.3.4 Consider the $DISTORTEDTkeepForthMARS$ example from the UCR-Hexagon archive [49]. This dataset is of length 8184, with the first 4000 datapoints acting as a (positive only) training set. As shown in Fig. 20, if we run top-1 anomaly detection on this with all twenty-two features, the best anomaly is a false positive.

Moreover, note that the Matrix Profile (using the *leftMP*, as recommended in [49]) also fails here.

We extracted the training data and used the ideas in Sect. 5.3.4 to create two classes of data: normal (sampled from original data), and (synthetic) anomalous (note during this process,

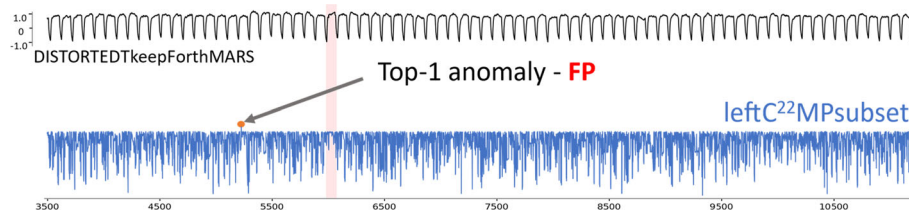


Fig. 20 Using C^{22} full, the top-1 anomaly for $DISTORTEDTkeepForthMARS$ is a false positive. The algorithm seems to have been confused by wandering baseline

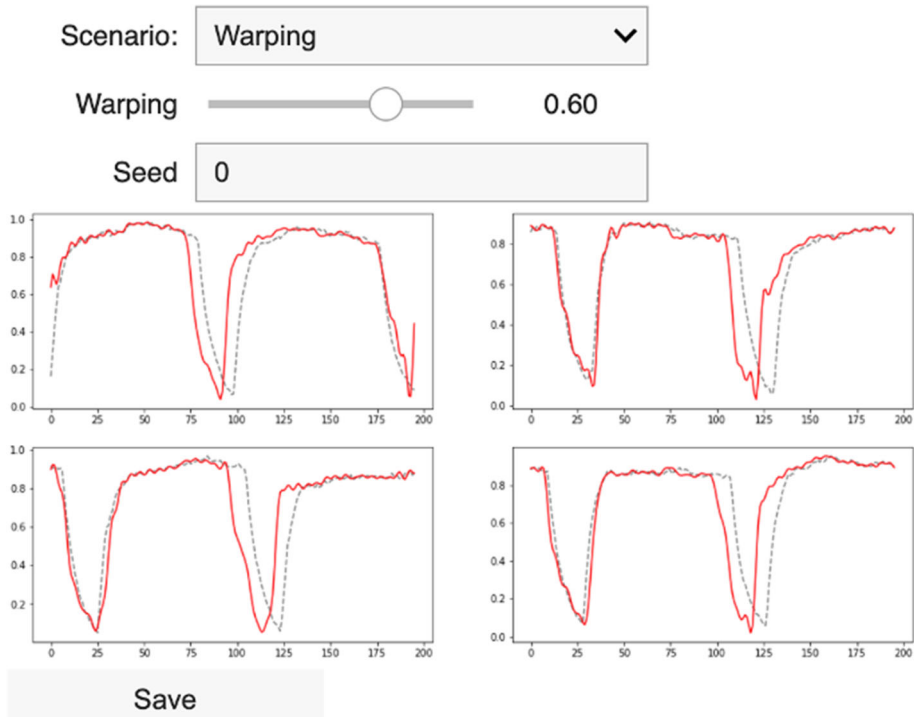


Fig. 21 A screen capture of a video showing adding warping distortion to DISTORTEDTkeepForthMARS data from the UCR-Hexagon datasets

we are learning the *constraints* that distinguish anomalous and normal behavior). We only considered the distortions of *warping*, *spike*, *noise*. The entire human-annotation session only took about 1 min, and a complete video of the process is archived at [10]. Figure 21 shows a screenshot of this tool. Here the user can simply select the type of distortion (in this example *warping*), then adjust the amount of distortion they would like to be added to the data using the slider. In this example, we can see the original data in gray (dashed lines) and synthetically distorted samples in red.

We then used the algorithm outlined in Sect. 5.3.3 to learn a feature set for this problem, the algorithm suggested using features {22, 10, 14, 12}. As shown in Fig. 22, this feature set *does* find the sole true positive.

We repeated the same process with other UCR-Hexagon datasets, to see if learned feature sets differed from dataset to dataset. This *is* the case. For example, for DISTORTED-CIMIS44AirTemperature2 we learned the feature set {14,15,17,7,10}, and this small feature set succeeded where C²²full had failed.

We will not evaluate all the UCR-Hexagon datasets this way. It would clearly not be fair to rival methods to compare against an algorithm that is availing of human help. However, these examples speak to the *expressiveness* of the weighted C²²MP and the relative ease of setting for good weight.

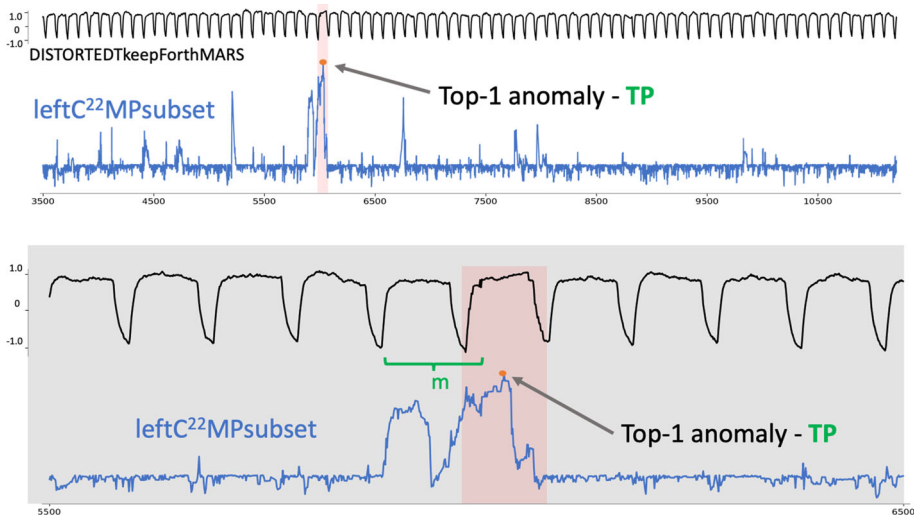


Fig. 22 *Top* Top anomaly for DISTORTEDTkeepForthMARS using C^{22}_{subset} is a true positive. *Bottom* A zoom-in of the top anomaly for DISTORTEDTkeepForthMARS using C^{22}_{subset}

6.9.1 How fast is ORR?

The previous experiments suggested that we can compute the $leftC^{22}MP$ efficiently. Here, we will more formally test this. We created a random walk time series of increasing lengths and compared the time for ORR to the time for the brute force approach. We choose random walk because it is the worst case for us, the lack of significant anomalies means that the early-abandoning technique is not as efficient as in a dataset that has anomalies.

As Fig. 23 shows, ORR is extremely fast. By the time we consider 64,000 datapoints, it is ~ 185 times faster than brute force. Its throughput is about 880 Hz, which is faster than the arrival rate of almost all accelerometers/medical telemetry, etc. In the plot, it is difficult to tease apart the line for ORR, and the line for *just* the feature profile extraction step. In fact, about $\frac{2}{3}$ of the time needed for ORR is in the feature extraction step. This suggests that it may be worth further optimizing that step. We first use the original authors MATLAB code [33] and found that with some simple optimizations, we could make it two

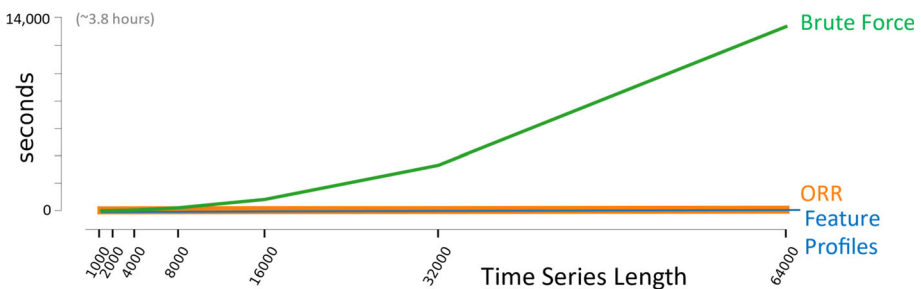


Fig. 23 A comparison of ORR and the brute force approach to compute the $leftC^{22}MP$. The blue curve shows just the time to do the C^{22} feature extraction

orders of magnitude faster (our faster code is freely available [10]), and we suspect further optimizations are possible.

7 Conclusions and future work

We have introduced a novel representation, C²²MP, and demonstrated that it provides state-of-the-art results for anomaly detection. It produces the highest published scores on the HEX/UCR benchmark, and it has been deployed in a biomedical lab to aid research. Moreover, while C²²MP is a generic TSAD technique, we have shown on the CCT dataset that it is able to beat domain specialized algorithms.

Here, we confined our attention to just anomaly detection. In future work, we plan to generalize the C²²MP to some of the other MP primitives, including motifs, chains and snippets. This will not be trivial. Anomaly detection requires only the discovery of the *highest* values in the C²²MP, and that is amiable to early-abandoning search. The other primitives require finding the *lowest* values in the C²²MP, a much more difficult task to accelerate.

Acknowledgements We thank all the creators of the data sets used in this work and the original authors of catch22, who were very helpful with their time [33].

Author contributions ST involved in algorithm design, writing, and implementation. YL involved in design of algorithm comparison measure. RW involved in optimization of code. TVAS involved in design of normalization algorithm. HDC involved in design of biological (mouse) experiments. RM involved in design of DNA experiments. EK involved in writing and editing.

Funding Funding was provided by gifts from Google, Mitsubishi and by NSF Award 2103976.

Data availability C²²MP (2022) Supporting webpage: sites.google.com/view/c22mp/home.

Declarations

Competing interests The authors declare that they have no competing interests.

References

1. Agrahari R et al (2022) Assessing feature representations for instance-based cross-domain anomaly detection in cloud services univariate time series data. *IoT* 3(1):123–144
2. Alzantot M, Chakraborty S, Srivastava M (2017) Sensegen: a deep learning architecture for synthetic sensor data generation. In: 2017 IEEE international conference on pervasive computing and communications workshops (PerCom Workshops), pp 188–193. IEEE
3. Aminifar F et al (2022) A review of power system protection and asset management with machine learning techniques. *Energy Syst* 13(4):855–892
4. Audibert J, Marti S, Guyard F, Zuluaga MA (2021) From univariate to multivariate time series anomaly detection with non-local information. In: International workshop on advanced analytics and learning on temporal data, pp 186–194. Springer, Cham
5. Audibert J, Michiardi P, Guyard F, Marti S, Zuluaga MA (2022) Do deep neural networks contribute to multivariate time series anomaly detection? arXiv preprint <https://arxiv.org/abs/2204.01637>
6. Audibert J, Michiardi P, Guyard F, Marti S, Zuluaga MA (2020) USAD: unsupervised anomaly detection on multivariate time series. In: Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining, pp 3395–3404
7. Boniol P, Linardi M, Roncallo F, Palpanas T, Meftah M, Remy E (2021) Unsupervised and scalable subsequence anomaly detection in large data series. *VLDB J* 30(6):909–931
8. Brophy E, Wang Z, She Q, Ward T (2021) Generative adversarial networks in time series: A survey and taxonomy. arXiv preprint <https://arxiv.org/abs/2107.11098>

9. C22MP (2022) Supporting webpage: sites.google.com/view/c22mp/home
10. Dau HA et al (2019) The UCR time series archive. *IEEE/CAA J Automatica Sinica* 6(6):1293–1305
11. Fährmann D, Damer N, Kirchbuchner F, Kuijper A (2022) Lightweight long short-term memory variational auto-encoder for multivariate time series anomaly detection in industrial control systems. *Sensors* 22(8):2886
12. Fengming Z, Shufang L, Zhimin G, Bo W, Shiming T, Mingming P (2017) Anomaly detection in smart grid based on encoder-decoder framework with recurrent neural network. *J China Univ Posts Telecommun* 24(6):67–73
13. Geiger A, Liu D, Alnegheimish S, Cuesta-Infante A, Veeramachaneni K (2020) Tadvan: time series anomaly detection using generative adversarial networks. In: 2020 IEEE international conference on big data (Big Data), pp 33–43. IEEE
14. Goh J, Adepu S, Junejo KN, Mathur A (2016) A dataset to support research in the design of secure water treatment systems. In: International conference on critical information infrastructures security, pp 88–99. Springer
15. Haq IU, Lee BS (2023) TransNAS-TSAD: harnessing transformers for multi-objective neural architecture search in time series anomaly detection. arXiv preprint <https://arxiv.org/abs/2311.18061>
16. Huet A, Navarro JM, Rossi D (2022) Local evaluation of time series anomaly detection algorithms. In: Proceedings of the 28th ACM SIGKDD, pp 635–645
17. Hundman K, Constantinou V, Laporte C, Colwell I, Soderstrom T (2018) Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. In: Proceedings of 24th ACM SIGKDD, pp 387–395
18. Idé T (2006) Why does subsequence time-series clustering produce sine waves? In: Knowledge discovery in databases: PKDD 2006: 10th European conference on principles and practice of knowledge discovery in databases Berlin, Germany, Proceedings, vol 10, pp 211–222. Springer, Berlin
19. Jackson TD et al (2021) The motion of trees in the wind: a data synthesis. *Biogeosciences* 18(13):4059–4072
20. Keogh E, Lin J (2005) Clustering of time-series subsequences is meaningless: implications for previous and future research. *Knowl Inf Syst* 8:154–177
21. Kravchik M, Shabtai A (2021) Efficient cyber attack detection in industrial control systems using lightweight neural networks and PCA. *IEEE Trans Depend Secure Comput* 19(4):2179–2197
22. Lai KH, Zha D, Xu J, Zhao Y, Wang G, Hu X (2021) Revisiting time series outlier detection: Definitions and benchmarks. In: 35th Conference on NeurIPS datasets and benchmarks track
23. Li D, Chen D, Jin B, Shi L, Goh J, Ng SK (2019) MAD-GAN: multivariate anomaly detection for time series data with generative adversarial networks. In: Artificial neural networks and machine learning—I-CANN 2019: text and time series: 28th international conference on artificial neural networks, Munich, Germany, Proceedings, part IV, pp 703–716. Springer, Cham
24. Liu HY, Gao ZZ, Wang ZH, Deng YH (2022) Time series classification with shapelet and canonical features. *Appl Sci* 12(17):8685
25. Loh WY (2011) Classification and regression trees. *Wiley Interdiscip Rev Data Min Knowl Discov* 1(1):14–23
26. Lu Y, Wu R, Mueen A, Zuluaga MA, Keogh E (2022) Matrix profile XXIV: scaling time series anomaly detection to trillions of datapoints and ultra-fast arriving data streams. In: Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining, pp 1173–1182
27. Lubba CH, Sethi SS, Knaute P, Schultz SR, Fulcher BD, Jones NS (2019) catch22: CAnonical time-series CHaracteristics. *Data Min Knowl Disc* 33(6):1821–1852
28. Lauer J, Zhou M, Ye S, Menegas W, Nath T, Rahman MM, Di Santo V, Soberanes D, Feng G, Murthy VN, Lauder G (2021) Multi-animal pose estimation and tracking with DeepLabCut. *BioRxiv*
29. MacQueen J (1967) Classification and analysis of multivariate observations. In: 5th Berkeley symposium on mathematics and statistics and probability, pp 281–297. University of California, Los Angeles
30. Marimon X, Traserra S, Jiménez M, Ospina A, Benítez R (2022) Detection of abnormal cardiac response patterns in cardiac tissue using deep learning. *Mathematics* 10(15):2786
31. Munir M, Siddiqui SA, Dengel A, Ahmed S (2018) DeepAnT: a deep learning approach for unsupervised anomaly detection in time series. *IEEE Access* 19(7):1991–2005
32. Nakamura T, Imamura M, Mercer R, Keogh E (2020) Merlin: parameter-free discovery of arbitrary length anomalies in massive time series archives. In: 2020 IEEE ICDM, pp 1190–1195
33. Park D, Hoshi Y, Kemp CC (2018) A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder. *IEEE Robot Autom Lett* 3(3):1544–1551
34. Ren H, Xu B, Wang Y, Yi C, Huang C, Kou X, Xing T, Yang M, Tong J, Zhang Q (2019) Time-series anomaly detection service at microsoft. In: Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, pp 3009–3017

35. Rewicki F, Denzler J, Niebling J (2022) Is it worth it? An experimental comparison of six deep-and classical machine learning methods for unsupervised anomaly detection in time series. arXiv preprint <https://arxiv.org/abs/2212.11080>
36. Saarela M, Jauhiainen S (2021) Comparison of feature importance measures as explanations for classification models. *SN Appl Sci* 3:1–2
37. Thompson DW (1917) On growth and form. Cambridge University Press
38. Tuli S, Casale G, Jennings NR, Tranad: deep transformer networks for anomaly detection in multivariate time series data. arXiv preprint <https://arxiv.org/abs/2201.07284>
39. Turowski M et al. (2022) Modeling and generating synthetic anomalies for energy and power time series. In: *Proceedings of the 13th ACM e-Energy*, pp 471–484
40. Wang R, Liu C, Mou X, Guo X, Gao K, Liu P, Wo T, Liu X (2022) Deep contrastive one-class time series anomaly detection. arXiv preprint <https://arxiv.org/abs/2207.01472>
41. Wen Q, Sun L, Yang F, Song X, Gao J, Wang X, Xu H (2020) Time series data augmentation for deep learning: a survey. arXiv preprint <https://arxiv.org/abs/2002.12478>
42. Wu R, Keogh E (2021) Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE TKDE*
43. Yairi T, Kato Y, Hori K (2001) Fault detection by mining association rules from house-keeping data. In: *Proceedings of the 6th international symposium on artificial intelligence, robotics and automation in space*, vol 18, p 21. Citeseer
44. Yankov D, Keogh E, Rebbapragada U (2008) Disk aware discord discovery: finding unusual time series in terabyte sized datasets. *Knowl Inf Syst* 17:241–262
45. Yoon J, Jarrett D, Van der Schaar M (2019) Time-series generative adversarial networks. In: *Advances in neural information processing systems*, vol 32
46. Zhang C, Kuppannagari SR, Kannan R, Prasanna VK (2018) Generative adversarial network for synthetic time series data generation in smart grids. In: *2018 IEEE international conference on communications, control, and computing technologies for smart grids (SmartGridComm)*, pp 1–6. IEEE
47. Zhu Y, Yeh CC, Zimmerman Z, Kamgar K, Keogh E (2018) Matrix profile XI: SCRIMP++: time series motif discovery at interactive speeds. In: *2018 IEEE ICDM*, pp 837–846

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Sadaf Tafazoli received her BS and MS in Computer Science from IUST (Iran University of Science and Technology) in 2011 and 2014, respectively, and obtained her PhD in computer science from the University of California, Riverside, in 2023. She specializes in leveraging her research focus on text, image, event, and time series data mining, coupled with machine learning techniques, to analyze a wide array of datasets. Her expertise spans various industries such as oil and gas, mining, pharmaceuticals, and renewable energy production.



Yue Lu received her PhD in Computer Science at the University of California, Riverside in 2023. Her research interests include time series analysis, data mining and machine learning.



Renjie Wu is currently a PhD candidate in Computer Science at the University of California, Riverside. He received his B.S. degree in Computer Science and Technology from Harbin Institute of Technology at Weihai in 2017. His research interests include time series analysis, data mining and machine learning.



Thirumalai Vinjamoor Akhil Srinivas received his MS from UCR in 2023. He specializes in data mining and machine learning primarily in the automotive industry.



Hannah Dela Cruz is currently pursuing her B.S. in Microbiology at the University of California, Riverside. Her research interests include applying computer vision and data mining techniques to extract new insights from biomedical imaging datasets, which will help facilitate pre-clinical drug research and development.



Ryan Mercer received a BS in electrical engineering from the University of California, Los Angeles, in 2014 and a PhD in computer science from the University of California, Riverside, in 2023. He has applied his research interests in time series data mining and machine learning to a diverse set of industries: automotive, finance, and renewable energy production and storage.



Eamonn Keogh is currently a distinguished professor of computer science with the University of California, Riverside. His research interests include time series, data mining, and computational entomology.